

CheckMate – Projektdokumentation

Projekt: CheckMate

Team: Thanh Hien Nguyen, Jarmila Dauth

Modul: Web-Technologien (WIN)

Studiengang: Wirtschaftsinformatik (B. Sc.)

Technologien: Vue.js 3, Spring Boot, Auth0, GitHub Pages, Render, MariaDB

Spezifikation und Design

Ziel der Applikation

CheckMate ist eine webbasierte Matching-Plattform für Studierende, die Study-Partner:innen oder Tutor:innen suchen.

Ziel der Applikation ist es, Studierende anhand gemeinsamer Fachinteressen und persönlicher Präferenzen effizient zusammenzubringen und eine einfache Kontaktaufnahme zu ermöglichen.

Die Anwendung verbindet klassische Katalogansichten mit Filterfunktionen und einem interaktiven Ja-/Nein-Entscheidungssystem zur Suche nach passenden Studypartnern. Studierende werden dabei einzeln angezeigt und können nach jedem Profil entscheiden, ob sie Interesse an einem Match haben oder nicht. Nach jeder Entscheidung erscheint automatisch ein neuer Student, den man kennenlernen kann.

Ziel ist es, die Suche nach geeigneten Lernpartnern einfach, übersichtlich und intuitiv zu gestalten und so den Aufbau von effektiven Lernbeziehungen zu fördern.

Projektziele

- Studierende beim Finden von Lernpartner:innen unterstützen
- Tutor:innen eine Plattform zur Präsentation ihrer Angebote bieten
- Eine einfache, schnelle Filterung nach Fach und Name ermöglichen
- Rollenbasierte Nutzung (Studierende, Tutor:innen, Administrator:innen)
- Sichere Authentifizierung und Autorisierung der Benutzer:innen
- Moderne, responsive Benutzeroberfläche

Personas

Persona 1: Lisa Müller – Studentin (Bachelor)

Alter: 21 Jahre

Studiengang: Maschinenbau im 2. Semester an der HTWG Konstanz

Wohnsitz: Böhringen

Familiensituation: Lebt mit ihrem Freund

Eigenschaft: sehr Introvertiert Ziel: Eine*n Lernpartner*in für ein Maschinenbau-Modul finden, um sich gemeinsam auf die Prüfung vorzubereiten. Zusätzlich sucht sie eine*n Tutor*in für ein weiteres Modul, der/die sie bei Bedarf flexible unterstützen kann.

Sie möchte schnell sehen, wer ähnliche Fächer belegt, und bevorzugt eine visuelle, einfache Navigation statt langer Suchformulare.

Persona 2: Max Schneider – Tutor

Alter: 27 Jahre

Studiengang: Architektur im 9. Semester an der HTWG Konstanz

Wohnsitz: Konstanz

Familiensituation: Lebt alleine

Eigenschaft: extrovertiert, sehr gut im Studium Ziel: Will durch seine Tätigkeit als Tutor auf CheckMate nebenher etwas zusätzliches Geld verdienen, indem er anderen Student*innen in verschiedenen Modulen hilft, die er schon belegt und bestanden hat. Das Geld soll in ein neues Sofa fließen.

Max gibt Nachhilfe in Mathematik und Modellieren.

Er möchte seine Angebote übersichtlich präsentieren und von Studierenden direkt kontaktiert oder gebucht werden können.

Persona 3: Admin

Ein Administrator verwaltet Tutor:innen und überwacht die Plattform.

Er benötigt Zugriff auf administrative Funktionen wie das Anlegen oder Löschen von Tutor:innen.

Use-Case-Übersicht

Use Case Übersicht



Die Applikation unterstützt folgende zentrale Use Cases:

- Benutzer anmelden / abmelden
- Study-Partner:innen durchsuchen (Swipe-Ansicht)
- Tutor:innen durchsuchen (Katalog-Ansicht)
- Filtern nach Name und Fach
- Filter zurücksetzen
- Tutor:innen kontaktieren
- Tutor:innen buchen
- Tutor:innen anlegen (Admin)
- Tutor:innen löschen (Admin)

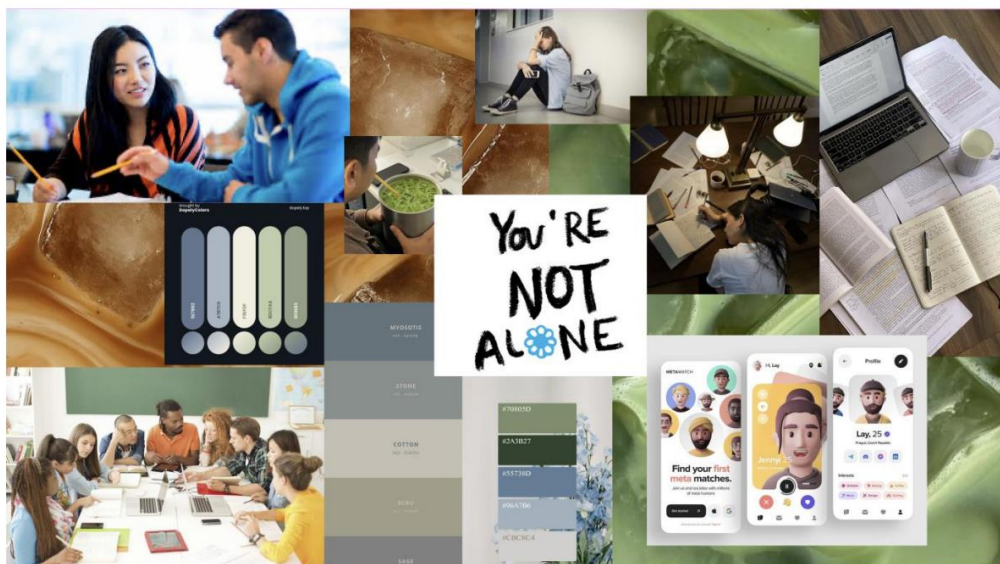
Design

Das visuelle Design orientiert sich an einer modernen, klaren und studierendenfreundlichen Gestaltung.

- Gestaltungsprinzip: übersichtlich, intuitiv, interaktiv
- Farbschema: natürliche Grün- und Beige-Töne
- Schrift: serifenlose Standardschrift (Web-optimiert)
- Responsive Design: optimiert für Desktop und mobile Endgeräte
- Mockups/Wireframes: in Figma erstellt

(Mockups mussten laut Aufgabenstellung nicht nachträglich angepasst werden.)

Moodboard

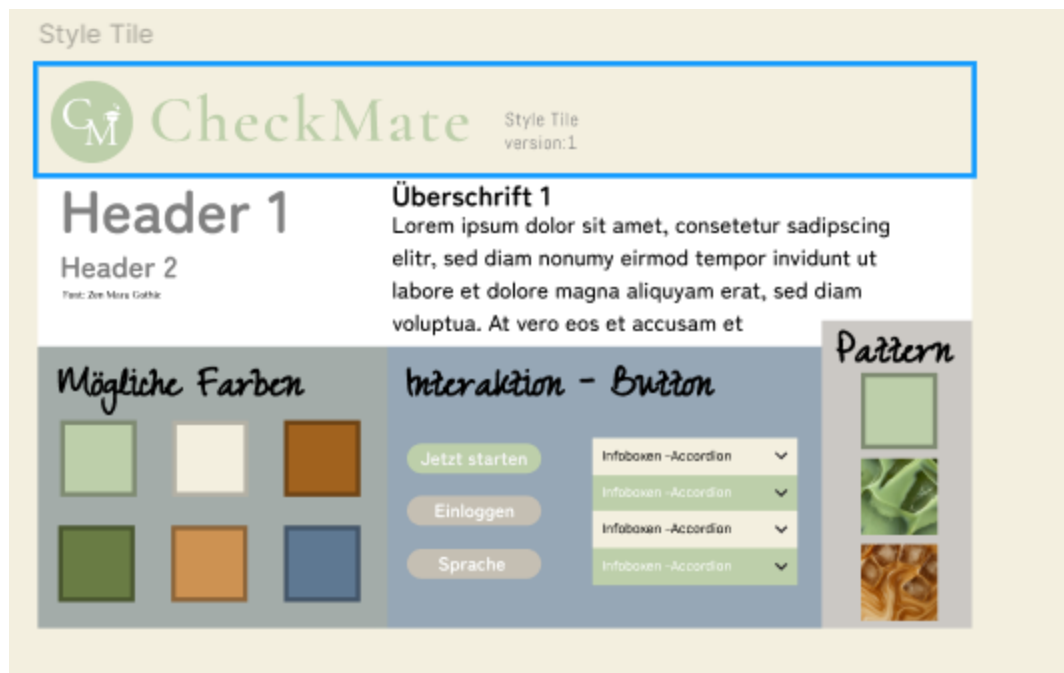


<https://www.figma.com/dacion/X79Gni776D7hRktGnIE/CheckMate?node-id=0,1&t=IEanfIDMDnfi#TMo-1>

Das Moodboard soll auf den ersten Blick das Thema Studieren sowie ein Gefühl von Verständnis, Zusammenhalt und gegenseitiger Unterstützung vermitteln. Im Mittelpunkt stehen Situationen des gemeinsamen Lernens, des Austauschs und der Konzentration, die den Alltag von Studierenden realistisch widerspiegeln.

Die verwendeten Bilder zeigen daher sowohl Lernszenarien (allein und in Gruppen) als auch ruhige, persönliche Momente, um die unterschiedlichen emotionalen Zustände im Studienalltag abzubilden – von Motivation und Fokus bis hin zu Überforderung. Dadurch entsteht ein authentisches Gesamtbild des studentischen Lebens.

Style Tile



Wir haben uns für diese Farbpalette entschieden, da die Anwendung bewusst einfach, übersichtlich und farblich zurückhaltend gestaltet werden soll. Die ruhigen Grundfarben unterstützen die Konzentration und sorgen für eine angenehme, nicht überladene Nutzeroberfläche.

Inhaltlich sind manche Farben gedanklich an Matcha und Kaffee angelehnt, da beide Getränke einen starken Bezug zum studentischen Alltag haben und unterschiedliche Stimmungen symbolisieren. Diese Inspiration spiegelt sich nicht nur in den Farben, sondern auch in den verwendeten Pattern wider.

Die Matcha-Elemente stehen für einen sanften Einstieg: Matcha wird als mild, süßlich und zugänglich wahrgenommen und symbolisiert Studierende, die sich noch am Anfang befinden oder erste Erfahrungen sammeln.

Die Kaffee-Elemente hingegen wirken dunkler und kräftiger. Sie stehen für Reife, Erfahrung und einen routinierteren Umgang mit dem Studium – vergleichbar mit dem bitteren Geschmack von Kaffee, den man oft erst mit der Zeit schätzen lernt.

Das Zusammenspiel aus Farben und Pattern unterstützt somit nicht nur die Ästhetik, sondern auch die inhaltliche Orientierung innerhalb der Applikation.

Implementierung

Benutzer anmelden / abmelden

Die Authentifizierung erfolgt über Auth0 im Frontend.

Geschützte Routen sind im Router (index.js) mit einem authGuard abgesichert, sodass nur authentifizierte Benutzer:innen Zugriff auf bestimmte Bereiche (z. B. Student-, Tutor- oder Admin-Bereich) erhalten.

Nach erfolgreichem Login wird ein Access-Token bezogen, das für den Zugriff auf geschützte Backend-Endpunkte genutzt wird.

Die Benutzerrolle (z. B. ADMIN) wird serverseitig bestimmt und im Frontend zur Steuerung der Benutzeroberfläche verwendet (z. B. Anzeige administrativer Funktionen).

Study-Partner:innen durchsuchen

Der Use Case Study-Partner:innen durchsuchen ist im Frontend über die View StudentCatalog.vue realisiert.

Dabei werden Studierendenprofile aus dem Backend geladen und nacheinander angezeigt.

Die Darstellung erfolgt in einer Ja/Nein-Entscheidungslogik, bei der Profile per Like oder Dislike bewertet werden können.

Die Steuerung der Anzeige erfolgt clientseitig, ohne dass für jede Entscheidung eine neue Backend-Anfrage notwendig ist.

Die zugehörigen Daten werden über den Backend (StudentController.java) bereitgestellt.

Tutor:innen durchsuchen

Die Katalog-Ansicht der Tutor:innen ist über die Komponente TutorCatalog.vue umgesetzt. Diese lädt die verfügbaren Tutor:innen aus dem Backend und übergibt sie an TutorList.vue, welche für die Darstellung der einzelnen Tutor-Cards verantwortlich ist.

Jede Tutor-Card zeigt relevante Informationen wie Name, Fachgebiet und Semester an. Durch Auswahl einer Tutor-Card wird über das Routing (/tutor/:id) zur Detailansicht TutorDetail.vue navigiert.

Die benötigten Tutor-Daten werden serverseitig durch den TutorController.java bereitgestellt.

Filtern nach Name und Fach

Ein wiederverwendbarer Filter-Component ermöglicht:

- Suche nach Namen
- Filterung nach Fach
- Zurücksetzen der Filter

Die Filterung erfolgt vollständig clientseitig über bereits geladene Daten.

Tutor:innen kontaktieren

Der Use Case *Tutor:innen kontaktieren* wird über die Nachrichten-Ansicht Messages.vue realisiert.

Nach Auswahl eines Tutors wird der/die Studierende über das Routing zur Nachrichtenansicht weitergeleitet, in der die Kommunikation stattfindet.

Die Navigation zur Nachrichtenfunktion erfolgt ausschließlich für authentifizierte Benutzer:innen.

Tutor:innen buchen

Tutor:innen können für bestimmte Zeiträume gebucht werden.

Dabei werden verfügbare Termine geladen und Überschneidungen mit bestehenden Buchungen verhindert.

Die Buchung kann dem Warenkorb hinzugefügt werden.

Tutor:innen anlegen (Admin)

Administrator:innen können neue Tutor:innen anlegen.

Die Erfassung der Daten erfolgt über ein Formular im Frontend, die Übermittlung an das Backend erfolgt per **POST-Request**.

Die Verarbeitung und Persistierung der Daten ist im TutorController.java implementiert. Nach erfolgreichem Anlegen wird der neue Tutor ohne Seitenneuladen im Frontend angezeigt.

Tutor:innen löschen (Admin)

Der Use Case *Tutor:innen löschen* steht ausschließlich Administrator:innen zur Verfügung. Die Löschung wird im Backend über den TutorController.java durchgeführt.

Nach erfolgreicher Entfernung eines Tutors wird die Änderung unmittelbar im Frontend reflektiert, sodass der gelöschte Eintrag nicht mehr in der Katalog-Ansicht erscheint.

Bereitstellung: Links/URLs der Projekte

Frontend (GitHub Pages):

<https://htwg-in-schneider.github.io/frontend-checkmate/>

Backend (Render):

<https://backend-checkmate-zovd.onrender.com>

Figma

<https://www.figma.com/design/XZ9GzqiZ26P7hRekzGgNIF/CheckMate?node-id=0-1&p=f&t=Sq7BTybJVRPEqnIU-0>

Testzugänge

Admin

- Benutzername: j.dauth@outlook.com
- Passwort: Checkmate123
- Login über Auth0

Student

- Benutzername: Thanhhiendang521@gmail.com
- Passwort: Checkmate123.
- Login über Auth0

Tutor

- Benutzername: Thanhhiendang521@yahoo.de
- Passwort: Checkmate123.
- Login über Auth0